

AIにただ任せるだけでテストがうまくいくのでしょうか。——答えはNoです。テストに対する理解があつてはじめて、AIにテストをうまく任せられます。本章では、従来のテストについての理解を深め、そのうえでAIの使いどころや使いこなすための勘所を整理します。

第3章

プロダクト品質を保証するためのテストとAI活用

従来のテストを理解し、
適切にAIに任せよう

Author 村穂 紀成(むらほ けいなり) オーティファイ株式会社

X(Twitter) @cazu_PR

本章では、プロダクトの品質を保証するためのテストにおいて、AIをどのように活用できるかをお伝えします。本章で説明しているテストは、システムテストやE2Eレベルのテストを想定しています。

AIを使いこなすには、AIにさせたい仕事を人間が理解しておく必要があります。そうしないと、AIのアウトプットを制御することも評価することもできないからです。そのため、本章の前半ではAIの話ではなく、プロダクトの品質を保証するためのテストについて説明します。そして、後半ではテストでのAI活用について説明します。

AIを活用する前に 押さえておきたいテストのこと

✓ **プロダクトの品質を保証する
テストを作るには仕様書だけでは足りない**

✗ **仕様書をそのままテストケースにしては
いけない**

仕様書の状態にもよりますが、一般的に、仕様書をそのままチェックリスト化したようなテストケースはよくないとされています(アジャイル開発を行っている方は「仕様書」を「バックログ」と読み替えていただいて差し支えありません)。その理由を3つ挙げます。

1つ目は、仕様書には、前提条件の記載漏れや、要件間の矛盾が残っていることがあるためです。このような仕様書をそのままテストの正解とす

ると、仕様書の誤りまで「正しい」ものとして扱ってしまいます。レビューの段階で除かれていることもあります。そうでない場合もあるため注意が必要です。

2つ目は、仕様書には、要件を満たすための動作を実現する方法が中心に書かれており、境界値、無効な入力、状態の組み合わせといった不具合を発見しやすいところ書かれていないケースが多いからです。

3つ目は、仕様書に書かれない前提が存在するためです。その業界で当然とされる慣習、法規制、求められる信頼性の水準など、テストの内容を考えるうえで必要な情報が仕様書の外にも存在する場合があります。

これらの理由から、テストケースを考える際には、仕様書を鵜呑みにするのではなく、書かれていることを分析したり、上位の要求事項や業界の前提を考慮したりする必要があります。

✗ テストレベルとテストタイプを活用する

一口にテストと言っても、その対象や観点はさまざまです。

たとえば「テストをしてください」と言われたとき、ある人は「自分が実装した関数の戻り値が正しいことを確認する」と思うでしょう。ある人は「システムの振る舞いが要件定義書に書かれているとおりであることを確認する」と思うでしょう。ある人は「ユーザーが心地良く使えることを確認する」と思うでしょう。この

ように、テストといってもそれが指すものはとても幅広いです。

そのため、テストをするときは、テストレベル（ユニットテスト、統合テスト、E2Eテストなど）やテストタイプ（機能テスト、性能テスト、セキュリティテストなど）など、テストという幅広い空間のどこに着目しているのか、どこの活動をしているのかを意識することで、他人と認識を合わせやすくなったり、テストの充足さを評価しやすくなったりします。

🔧 テストケースの爆発を防ぐ手段も必要

テストケースは増やそうと思えばいくらでも増やすことができます。入力できる値は膨大で、組み合わせパターンも無数に存在するためです。そのため、収束させる手段が必要になります。

この後に説明する、同値分割やペアワイズ法といった、合理的にテストケースを減らすテスト技法を用いるアプローチや、発生確率と影響度からリスクを定義したうえで、リスクの高い領域に重点的にテストを絞り込むといったアプローチが必要になります。

☑ テスト技法の使い分け

🔧 テスト技法を活用して効果的にテストする

闇雲にテストケースを作ると、過剰な量のテストケースになったり、過小な量のテストケースになったり、不具合を見つけられないテストになったりするため、テストを設計するという考えが必要になります。

テストエンジニアがテストを設計する際、テスト技法を使用することがあります。テスト技法を使わなくてもテストは作れますが、技法を使うことで、不具合を見つけやすいテストケースを効率よく導出したり、増え過ぎたテストケースを根拠を持って削減したりできます。代表的なテスト技法と、それぞれがどんな困りごとに対して役立つかを説明していきます。

🔧 困りごと「テストケースが多過ぎる」

→同値分割やペアワイズ法

同値分割は、同等に処理される、または同じ結果を返すデータを同値クラスとしてまとめ、その同値クラスの代表値だけをテストすれば十分とする技法です。たとえば、年齢入力フォームで「18歳以上65歳未満は通常料金、65歳以上はシニア料金、18歳未満は利用不可」という仕様があるとします。このとき、18~64はすべて同じ振る舞いをする値なので、これらは同値クラスとなります。この同値クラスから1つの代表値（たとえば30）をテストすれば、通常料金のテストは十分とします。シニア料金や利用不可に対しても同値クラスの代表値を決めてテストします。こうすることで、何十件のテストをすることなく、それぞれの振る舞いの期待結果をカバーしたテストを数件で達成できます。同値分割はテストケースを合理的に削減したいときに有効な技法です。この後に説明する境界値テストと組み合わせる使用が多いです。

ペアワイズ法は、多数のパラメータの全組み合わせをテストすることが難しい場合に、任意の2因子間の組み合わせだけを網羅するようにテストケースを作るテスト技法です。たとえば、OS×ブラウザ×画面解像度×言語設定の全組み合わせは膨大ですが、ペアワイズ法を使えば組み合わせを大幅に減らしつつ、3因子以上の組み合わせもバランスよく導出できます。ペアワイズ法は手動でやるよりもPICTなどのツールを使ってテストケースを抽出することが多いです。

🔧 困りごと「不具合が見つからない」

→境界値テスト

不具合は値の境界付近に集中する傾向があります。境界値テストは、同値クラスの境界にある値をピンポイントでテストする技法です。先ほどの年齢の例で言えば、17、18、64、65が境界値です。境界を狙うことで、効率的に

不具合を発見できます。同値クラスで境界を見つけ出し、境界値テストでその境界をピンポイントに狙う、という組み合わせがよく用いられます。

❧ 困りごと「条件が複雑で整理できない」

→ デシジョンテーブルや状態遷移テスト

デシジョンテーブルは、複数の条件とその組み合わせによって結果が変わるシステムの要件を整理したり洗い出したりできる技法です。「会員ランクがゴールド以上、かつ購入金額が1万円以上、かつクーポン適用可能な場合に割引率は……」のようなビジネスルールが複雑に絡み合う場面で、条件の組み合わせと期待される結果を一覧化できます。組み合わせパターンがそれぞれテストケースとなります。

状態遷移テストは、状態とイベントによる状態の遷移を洗い出せる技法です。デシジョンテーブルと同様に複雑な要件を整理するときにも役立ちます。状態遷移「図」を書くことで、状態とイベントの相互作用をわかりやすく可視化できます。また、状態遷移「表」を作成することで、正常な遷移だけでなく不正な遷移を洗い出しやすくなります。



テスト技法はこれら以外にもさまざまなものがあります。テスト技法を学ぶにあたり、筆者が参考にした書籍を記載しておきます。

- ・『ソフトウェアテスト技法ドリル：テスト設計の考え方と実際』^{注1}
- ・『はじめて学ぶソフトウェアのテスト技法』^{注2}
- ・『ソフトウェアテスト技法練習帳～知識を経験に変える40問～』^{注3}

注1) 秋山 浩一 著、日科技連出版社、2010年
初版は現在絶版。第2版が2022年に刊行。

注2) リー・コーブランド 著、宗雅彦 訳、日経BP、2005年

注3) 梅津 正洋、竹内 亜未、伊藤 由真、浦山 さつき、佐々木 千絵美、高橋 理、武田 春恵、根本 紀之、藤沢 耕助、真鍋 俊之、山岡 悠、吉田 直史 著、技術評論社、2020年

テスト設計でAIを活用するときの 考え方や注意点

☑ 人間が学習する工程の必要性

❧ ステークホルダーとの対話は必要

すでに書いているように、テストケースを作る際は、仕様書以外にもさまざまな情報が必要になってきます。また、テスト技法を適用したとしても、導出したテストケースをどこまでテストするかについて、テスト担当者だけで決めることはリスクが高いです。テスト担当者だけで決めてしまった場合、ステークホルダーから「ここもテストしてくれると思っていたのに」や「そこはすでにテストを行ったのに」といったトラブルが起こります。プロジェクト固有の情報や、ステークホルダーのニーズは、AIに聞いても知ることはできません。そのため、ステークホルダーとの対話はこれまでどおり必要になってきます。

❧ いきなりAIに生成させない

自分自身がテスト対象やテストケースの導出方法を理解していなければ、AIが出力したテストケースの妥当性を判断できません。結果として、レビューの段階で「このテストケースで本当にいいのか？」を一から調べることになり、自分でテストケースを作るよりも時間がかかることがあります。

そのため、AIを活用するフェーズは2つあると考えています。1つは学習のフェーズ（テスト対象や技法についてAIと対話しながら理解を深める）、もう1つはアウトプットのフェーズ（理解した内容をもとにAIにテストケースを出力させる）。すでに理解していることのアウトプットにAIを使うのは非常に有効ですが、理解が伴わないままアウトプットだけを加速させると、後から大きな手戻りが発生する恐れがあります。

✓ ワークフローとコンテキストで 出力精度が安定し、評価もしやすくなる

テストケースを作成するにはさまざまな情報が必要です。これらすべてを1つのプロンプトに書き起こしてAIに伝えようとする、コンテキストウィンドウを超過して出力が不安定になる場合があります。また、規模の大きいタスクをAIに依頼すると、実行ごとの出力結果のばらつきが大きくなる場合があります。

これらの問題に対して有効なのが、コンテキストエンジニアリングを用いたアプローチです(本章ではコンテキストエンジニアリングを「AIへの情報の与え方を最適化すること」と定義します)。

🔧 1回で大量の処理をさせず段階的に処理させる

テストケースの作成というタスクの中には、さまざまな作業が含まれています。たとえば、次のような作業です。

1. テスト対象を特定する
2. テストする観点を検討する
3. 適切なテスト技法を選定する
4. 具体的なテストケースに展開する

さまざまな作業が含まれているということは、そのタスクの規模が大きいことを意味します。タスクの規模が大きいほど、AIの出力結果がブレやすくなるため、タスクを小さいタスクに分割することをオススメします。これにより出力結果のブレを抑えつつ、また、中間成果物が生成されることで、最終成果物の妥当性を評価しやすくなります。

🔧 情報の置き場所を設計する

「～してください」という指示と、その指示を受けて処理される情報や参照すべき情報は、性質が異なる情報です。これらは分けて管理したほうが扱いやすくなります。

たとえば、次のような情報と、テストケースを作成させるための指示の情報は分けて管理し

たほうがよいと考えています。

- ・テスト方針
- ・ステークホルダーの要求事項
- ・業界固有の規制や前提知識
- ・過去の不具合情報
- ・プロダクトの機能追加に応じて必要になっていくテストの観点

これらは指示の中に直接記述するのではなく、コンテキストとして一元管理し、指示の中では参照先だけを示すことで情報の管理がしやすくなります。



上記を実現できるツールとして、Autifyが提供している Autify Genesis を紹介します。

Autify Genesis はAIにタスクを実行させるためのワークフローを構築できます(図1)。ワークフローは複数のブロックをつなぎ合わせて作ることができ、ブロックにはそれぞれプロンプトを書き込むことができます。また、ユーザーがアップロードしたドキュメントやワークフローで生成された成果物を保管しており、これらをワークフローの中でAIに参照させることができます。

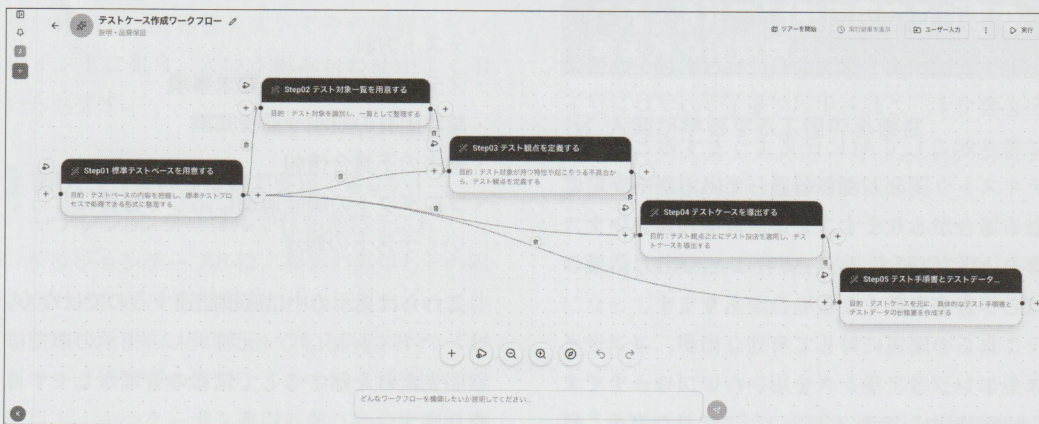
図1に映っているワークフローは、次のブロックで構成されています。

- ブロック1：ユーザーからアップロードされたテストベースを事前に定義した構造に変換する
- ブロック2：テストベースからテスト対象一覧を作成する
- ブロック3：テスト対象に適用するテスト観点を定義する
- ブロック4：テストケースを作成する
- ブロック5：テスト手順書とテストデータの仕様書を作成する

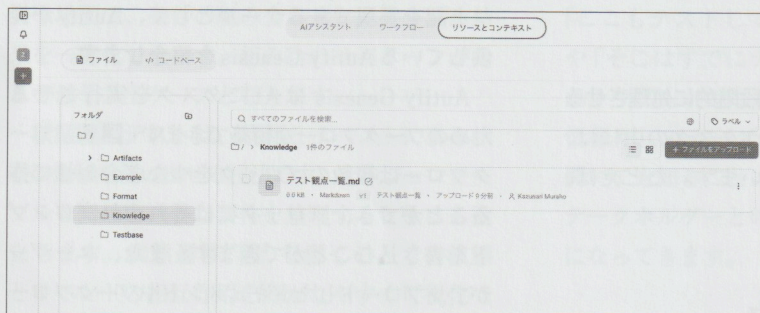
このワークフローでは、前のブロックの成果物を次のブロックのインプットに使っています。

ブロック3では、図2の画面にアップロード

▼図1 Autify Genesisのワークフロー構築画面(左から「ブロック1」「ブロック2」……)



▼図2 Autify Genesisのコンテキスト画面



高度な自動化をしたいがコードを書けない」という理由で手動テストにとどまっていた領域にも、自動化の選択肢が広がりました。

Autifyが提供している Autify Nexus というプラットフォームは、

E2Eの自動テストをローコードで実装できます。必要に応じて、JavaScriptやPlaywrightのコードを書けるため、より高度な実装も行えます。しかし前述のとおり、ローコードツールを利用するユーザーはコーディング経験がないことが多いため、これらの機能を十分に活用できない状況でした。

このような状況に対して、コーディングを支援してくれるAI機能が新たに実装されたことで、コーディングに慣れていないユーザーも、AIのサポートを借りて高度な実装を行えるようになりました。

図3の左に映っているのがAIと対話するための Autify NexusのUI、中央に映っているのがテスト対象のUI、右に映っているのがPlaywrightコードを入力するための Autify NexusのUIです。

されているテスト観点一覧のMarkdownファイルを参照するようにしています。

テスト自動化におけるAIの活用場面

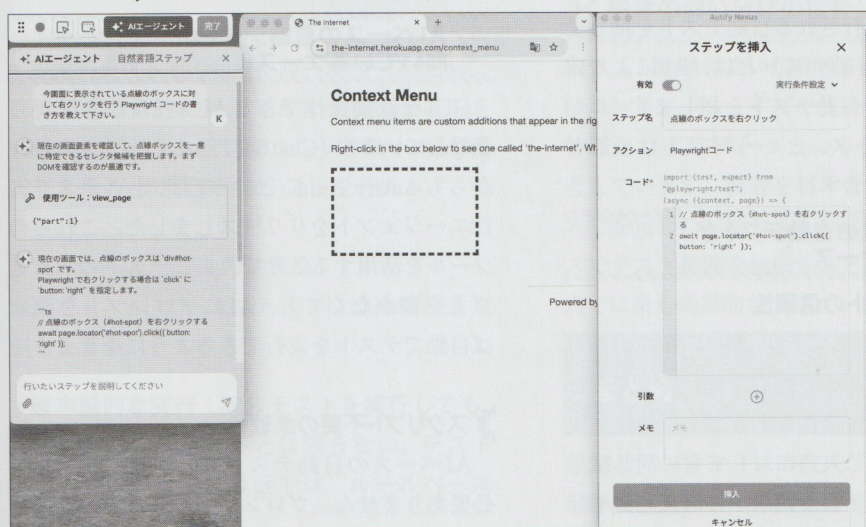
☑ ローコードツールとAIがもたらした変化

ローコードツールの登場により、自動化のハードルは大幅に緩和されました。

しかし、自動化したいことが増えたり、より壊れづらい実装を目指したりすると、コーディングが必要になる場合があります。ローコードツールを利用するユーザーはコーディングの経験がないことが多く、求めている自動化を行うのが難しい状況でした。

AIの登場により、コーディング経験がなくても、自然言語で意図を伝えることでコードを生成できるようになりました。その結果「より

▼図3 Autify NexusのAIを用いた支援機能



✓ 自動テストの失敗原因を調査させる

✎ E2Eテストの失敗調査はなぜ難しいのか

E2Eレベルの自動テストが失敗したとき、その原因を調査する作業は、ユニットテストやAPIテストに比べて難しいです。操作対象の範囲が広く、画面遷移やデータの状態変化が複雑に絡み合うためです。

さらに厄介なのは、失敗の原因が、失敗が発生した箇所にあるとは限らないということです。手前の操作で意図しない結果が生じており、それが数ステップ先で初めてエラーとして表面化するケースはよくあります。

たとえば、次のような流れを想像してください。

ステップ1：ログイン→OK

ステップ2：カートに商品を追加→OK

(※ここでデータが不正な状態に)

ステップ3：配送先を入力→OK

ステップ4：購入確認画面を表示→エラー発生！

この場合、エラーが出たのはステップ4ですが、原因はステップ2にあります。ステップ4のスクリーンショットだけを見ても原因はわかりません。

✎ 失敗までの過程も含めてAIに渡す

この問題に対してAIを活用する際は、失敗した箇所だけをAIに調べさせるのではなく、失敗までの一連の過程まで含めて調べさせるのが有効です。一連の流れを参照させることで、「どのステップでデータの状態が意図と異なったか」をAIが推測しやすくなり、原因特定の精度が向上します。

先ほど紹介したAutify Nexusは、テストの失敗原因をAIに調査させ修正まで行える機能も備わっています。以前は失敗した箇所が発生していたエラーメッセージのみを調査の対象にしていたましたが、根本的な原因の特定に至らないことがありました。その後、機能の改善が行われ、現在は、失敗原因を調査するにあたって、再度テストを実行し、失敗までの過程も調査範囲に含めることで、より正確な失敗原因を特定しやすくなりました。

テスト実行時のツールの使い分け

ここでは、ルールベースの自動テストと、AIベースの自動テストそれぞれの使い分けについて説明します。

ルールベースの自動テストとは、Playwright

やAutify Nexusのような、事前に定義されたとおりの手順で実行される自動テストを指します。AIベースの自動テストとは、AIによる確率論的に行われる自動テストを指します（ルールベース、AIベース、という呼び方は、筆者独自のものになります）。

✓ ルールベースの自動テストが向いているケース

決定論的なテストの信頼性

AIの精度が向上してきてはいるものの、テスト結果の信頼性という観点では、ルールベースの自動テストに優位性があります。結果が決定論的であり、同じ入力に対して常に同じ結果が返るためです。テスト結果が毎回変わる可能性がある、というだけで、その結果に対する信頼は揺らぎます。

また、プログラムで確実に判定できる処理を、わざわざAIのトークンを消費して実行するのは、コスト面でも合理的ではありません。

ルールベースが力を発揮する場面

ルールベースの自動テストが特に向いているのは、次のようなケースです。

- ・仕様が安定している機能：テストスクリプトが壊れづらく、投資対効果が高い
- ・合否判定が明確な検証：数値の一致、特定の要素の存在確認、ステータスコードの検証など、期待値が一意に定まるもの
- ・回帰テスト：リリースのたびに繰り返し実行するテストスイート

ルールベースの弱点：脆さと保守コスト

一方で、ルールベースの自動テストには脆さがあります。UIの変更やDOM構造の変化によって、テスト対象の機能に問題がなくてもテストが壊れることがあります。この保守コストは、仕様変更の頻度が高い機能ほど大きくなります。仕様が安定していない段階でルールベースのテストを大量に書くと、保守コストが利点を上回っ

てしまう場合があります。

✓ AIベースの自動テストが向いているケース

GUIを直接操作できるAIツールが各社から登場しています（ChatGPT Atlasなど）。AutifyからもAutify Aximoという自律型AIテスト実行エージェントをリリースしました。これらのツールを活用することで、自動テストのスク립トを書かなくても、AIにプロンプトを渡せば自動でテストを実行できるようになります。

スクリプト不要の柔軟性

AIベースの自動テストは、コーディングが必要ありません。プロンプトでテストの意図を伝えるだけで、AIがテストを実行します。GUIを操作するため、内部のDOM構造やセレクトタの変更には左右されにくいという特徴があります。

図4に映っているのはAutify Aximoが実際にテストを実行している様子です。ユーザーから受け取った「全ての必須項目が埋まっていない場合、送信するボタンを押下しても画面遷移しないことを確認してください。」という指示に対して、まず画面のスクリーンショットを撮って現在の画面の状態を確認し、さらに画面を操作しながらテスト対象を探しています。テスト対象が見つかったら、指示に沿ったテストを自動で行います。テスト対象が見つからない場合は、テストを終了したり、ユーザーに質問をしたりします。

AIベースが力を発揮する場面

この特性を活かせるのは、次のケースです。

- ・内部的な仕様変更が頻繁に発生する機能：DOM構造やセレクトタに依存するルールベースの自動テストだと保守コストが高くなるが、GUIを操作するAIベースであれば影響を受けないため保守コストが変化しない
- ・自動化前のカバレッジ拡大：テストケースをAIに与えることで、ルールベースの自動テ

ストの実装を待たずにテストのカバレッジを拡大できる

- ・探索的テストでの利用：ルールベース以外に、テストの目的に対してさまざまなテストを行いたいとき、適切なコンテキストをAIに与えることで探索的なテストを行える

AIベースのテストの注意点

ただし、AIベースのテストには次の注意点があります。

- ・確率論的な実行：同じテストを実行しても、毎回同じ操作になるとは限らない。テスト結果の信頼性という面では、ルールベースに劣る
- ・実行速度：ルールベースに比べて遅い。AIが画面を認識し判断するプロセスが入るため、大量のテストを短時間で回すことには向いていない
- ・専門知識が必要な領域での誤判定：業務固有のルールや専門用語が関わる場面、つまりAIが学習できていない知識が必要な場面では、誤判定リスクが高まる
- ・コスト：AIの呼び出し回数が増えるとトークンコストが積み上がる

✓ 両者の使い分けの指針

テスト結果に高い信頼性が求められる場面ではルールベース、すばやくテスト範囲を広げたい場面ではAIベースという使い分けがよいと考えています。両者を組み合わせ、ルールベースで安定した回帰テストを確保しつつ、AIベースでテスト範囲を拡張するという戦略が、現時点では最も効果的なアプローチと考えています。

まとめ

AIはテストケースの生成、自動テストのコード作成、失敗原因の調査など、多くの場面で力を発揮します。しかし、「何をテストすべきか」「なぜそれをテストするのか」「どこまでテストすれば十分か」という設計判断は、プロジェクトの文脈を理解している人間にしかできません。

AIを使いこなすために必要なのは、プロンプトの書き方だけではなく、テストのことを理解し、ステークホルダーと対話し、適切な戦略を立てたうえで、AIをその実行手段として活用すること。それが、AI時代におけるプロダクトの品質を保証するためのアプローチだと考えています。SD

▼図4 Autify AximoによるAIベースの自動テストの実行画面

